

CSS DOCUMENTATION

1 .CSS Introduction

- What is CSS?

CSS stands for Cascading Style Sheet. It's a style sheet language that determines how the elements/contents in the page are looked/shown. CSS is used to develop a consistent look and feel for all the pages.

CSS was developed and is maintained by the World Wide Web Consortium (W3C). It was first released on December 17, 1996. The CSS Working group currently working with different browser vendors to add/enforce the new feature/ specifications in all the browsers.

CSS enables the separation of the content from the presentation. This separation provides a lot of flexibility and control over how the website has to look like. This is the main advantage of using CSS.

CSS3 or Cascading Style Sheets Level 3 is the third version of the CSS standard that is used to style and format web pages. CSS3 incorporates CSS2 standard with some improvements over it. The main change in CSS3 is the inclusion of divisions of standards into different modules that makes CSS3 easier to learn and understand.

- The advantages of using CSS

The main advantages of CSS are given below:

- Separation of content from presentation - CSS provides a way to present the same content in multiple presentation formats in mobile or desktop or laptop.
- Easy to maintain - CSS, built effectively can be used to change the look and feel complete by making small changes. To make a global change, simply change the style, and all elements in all the web pages will be updated automatically.
- Bandwidth - Used effectively, the style sheets will be stored in the browser cache and they can be used on multiple pages, without having to download again.

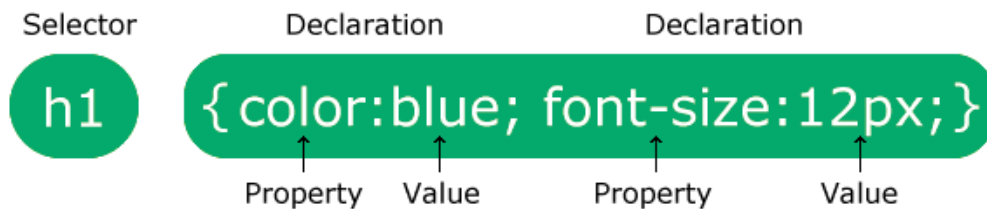
- The limitations of CSS

Disadvantages of CSS are given below:

- Browser Compatibility: Some style selectors are supported and some are not. We have to determine which style is supported or not using the @support selector).

- Cross Browser issue: Some selectors behave differently in a different browser).
- There is no parent selector: Currently, Using CSS, you can't select a parent tag.

2 .CSS Syntax



The selector points to the HTML element you want to style.

The declaration block contains one or more declarations separated by semicolons.

Each declaration includes a CSS property name and a value, separated by a colon.

Multiple CSS declarations are separated with semicolons, and declaration blocks are surrounded by curly braces.

Example:

In this example all <p> elements will be center-aligned, with a red text color:

```
p {
  color: red;
  text-align: center;
}
```

Example Explained

- p is a selector in CSS (it points to the HTML element you want to style: <p>).
- color is a property, and red is the property value.
- text-align is a property, and center is the property value.

3 .How To Add CSS in HTML ?

There are different ways to include a CSS in a webpage:

- Inline CSS:

Add inline styles to HTML elements(CSS rules applied directly within an HTML tag.): Style can be added directly to the HTML element using a style tag.

```
<h2 style="color:red;background:black">Inline Style</h2>
```

- Internal CSS:

Embed CSS with a style tag: A set of CSS styles included within your HTML page.

```
<style type="text/css">

/*Add style rules here*/

</style>
```

- External CSS:

An external file linked to your HTML document: Using link tag, we can link the style sheet to the HTML page.

```
<link rel="stylesheet" type="text/css" href="styles.css" />
```

- Import a stylesheet file:

(An external file imported into another CSS file): Another way to add CSS is by using the @import rule. This is to add a new CSS file within CSS itself.

```
@import "path/to/style.css";
```

4 .CSS Selectors

A CSS selector is the part of a CSS ruleset that actually selects the content you want to style. Different types of selectors are listed below.

- Universal Selector (*):

The universal selector works like a wildcard character, selecting all elements on a page. In the given example, the provided styles will get applied to all the elements on the page.

```
* {
  color: "green";
  font-size: 20px;
  line-height: 25px;
}
```

- Element Type Selector:

This selector matches one or more HTML elements of the same name. In the given example, the provided styles will get applied to all the ul elements on the page.

```
ul {  
  line-style: none;  
  border: solid 1px #ccc;  
}
```

- ID Selector:

This selector matches any HTML element that has an ID attribute with the same value as that of the selector. In the given example, the provided styles will get applied to all the elements having ID as a container on the page.

```
#container {  
  width: 960px;  
  margin: 0 auto;  
}
```

```
<div id="container"></div>
```

- Class Selector:

The class selector also matches all elements on the page that have their class attribute set to the same value as the class. In the given example, the provided styles will get applied to all the elements having ID as the box on the page.

```
.box {  
  padding: 10px;  
  margin: 10px;  
  width: 240px;  
}
```

```
<div class="box"></div>
```

- Descendant Combinator:

The descendant selector or, more accurately, the descendant combinator lets you combine two or more selectors so you can be more specific in your selection method.

```
#container .box {  
  float: left;  
  padding-bottom: 15px;  
}
```

```
<div id="container">
  <div class="box"></div>
  <div class="box-2"></div>
</div>
```

```
<div class="box"></div>
```

This declaration block will apply to all elements that have a class of box that is inside an element with an ID of the container. It's worth noting that the `.box` element doesn't have to be an immediate child: there could be another element wrapping `.box`, and the styles would still apply.

- Child Combinator:

A selector that uses the child combinator is similar to a selector that uses a descendant combinator, except it only targets immediate child elements.

```
#container > .box {
  float: left;
  padding-bottom: 15px;
}
```

```
<div id="container">
  <div class="box"></div>
  <div>
    <div class="box"></div>
  </div>
</div>
```

The selector will match all elements that have a class of `box` and that are immediate children of the `#container` element. That means, unlike the descendant combinator, there can't be another element wrapping `.box` it has to be a direct child element.

- General Sibling Combinator:

A selector that uses a general sibling combinator to match elements based on sibling relationships. The selected elements are beside each other in the HTML.

```
h2 ~ p {
  margin-bottom: 20px;
}
```

```

<h2>Title</h2>
<p>Paragraph example.</p>
<p>Paragraph example.</p>
<p>Paragraph example.</p>
<div class="box">
  <p>Paragraph example.</p>
</div>

```

In this example, all paragraph elements (<p>) will be styled with the specified rules, but only if they are siblings of <h2> elements. There could be other elements in between the <h2> and <p>, and the styles would still apply.

- **Adjacent Sibling Combinator:**

A selector that uses the adjacent sibling combinator uses the plus symbol (+), and is almost the same as the general sibling selector. The difference is that the targeted element must be an immediate sibling, not just a general sibling.

```

p + p {
  text-indent: 1.Sem;
  margin-bottom: 0;
}

```

```

<h2>Title</h2>
<p>Paragraph example.</p>
<p>Paragraph example.</p>
<p>Paragraph example.</p>

<div class="box">
  <p>Paragraph example.</p>
  <p>Paragraph example.</p>
</div>

```

The above example will apply the specified styles only to paragraph elements that immediately follow other paragraph elements. This means the first paragraph element on a page would not receive these styles. Also, if another element appeared between two paragraphs, the second paragraph of the two wouldn't have the styles applied.

- **Attribute Selector:**

The attribute selector targets elements based on the presence and/or value of HTML attributes, and is declared using square brackets.

```

input [type="text"] {

```

```
background-color: #444;  
width: 200px;  
}
```

```
<input type="text">
```

5 .CSS Comments

CSS comments are not displayed in the browser, but they can help document your source code. Comments are used to explain the code, and may help when you edit the source code at a later date.

A CSS comment is placed inside the <style> element, and starts with /* and ends with */:

Example

```
/* This is a single-line comment */
```

```
p {  
  color: red;  
}
```

```
/* This is  
a multi-line  
comment */
```

```
p {  
  color: red;  
}
```

6 .CSS Colors

Colors are specified using predefined color names, or RGB, HEX, HSL, RGBA, HSLA values.

- CSS Color Names

In CSS, a color can be specified by using a predefined color name.

Example:

```
<h1 style="background-color:DodgerBlue;">Hello World</h1>  
<p style="background-color:Tomato;">Lorem ipsum...</p>
```

- CSS RGB Colors

An RGB color value represents RED, GREEN, and BLUE light sources.

rgb(red, green, blue)

Each parameter (red, green, and blue) defines the intensity of the color between 0 and 255.

For example, rgb(255, 0, 0) is displayed as red, because red is set to its highest value (255) and the others are set to 0.

To display black, set all color parameters to 0, like this: rgb(0, 0, 0).

To display white, set all color parameters to 255, like this: rgb(255, 255, 255).

Example:

```
<h1>Specify colors using RGB values</h1>
```

```
<h2 style="background-color:rgb(255, 0, 0);">rgb(255, 0, 0)</h2>
```

```
<h2 style="background-color:rgb(0, 0, 255);">rgb(0, 0, 255)</h2>
```

- CSS RGBA Value

RGBA color values are an extension of RGB color values with an alpha channel - which specifies the opacity for a color.

An RGBA color value is specified with:

rgba(red, green, blue, alpha)

The alpha parameter is a number between 0.0 (fully transparent) and 1.0 (not transparent at all):

- CSS HEX Colors

A hexadecimal color is specified with: #RRGGBB, where the RR (red), GG (green) and BB (blue) hexadecimal integers specify the components of the color.

`#rrggbb`

Where rr (red), gg (green) and bb (blue) are hexadecimal values between 00 and ff (same as decimal 0-255).

Where rr (red), gg (green) and bb (blue) are hexadecimal values between 00 and ff (same as decimal 0-255).

For example, `#ff0000` is displayed as red, because red is set to its highest value (ff) and the others are set to the lowest value (00).

To display black, set all values to 00, like this: `#000000`.

To display white, set all values to ff, like this: `#ffffff`.

- **CSS HSL Colors**

HSL stands for hue, saturation, and lightness.

In CSS, a color can be specified using hue, saturation, and lightness (HSL) in the form:

`hsl(hue, saturation, lightness)`

Hue is a degree on the color wheel from 0 to 360. 0 is red, 120 is green, and 240 is blue.

Saturation is a percentage value. 0% means a shade of gray, and 100% is the full color.

Lightness is also a percentage. 0% is black, 50% is neither light or dark, 100% is white.

Saturation:

Saturation can be described as the intensity of a color.

100% is pure color, no shades of gray.

50% is 50% gray, but you can still see the color.

0% is completely gray; you can no longer see the color.

Lightness:

The lightness of a color can be described as how much light you want to give the color, where 0% means no light (black), 50% means 50% light (neither dark nor light) and 100% means full lightness (white).

Shades of Gray:

Shades of gray are often defined by setting the hue and saturation to 0, and adjust the lightness from 0% to 100% to get darker/lighter shades:

- CSS HSLA Value

HSLA color values are an extension of HSL color values with an alpha channel - which specifies the opacity for a color.

An HSLA color value is specified with:

hsla(hue, saturation, lightness, alpha)

The alpha parameter is a number between 0.0 (fully transparent) and 1.0 (not transparent at all)

7 .CSS Backgrounds

The CSS background properties are used to add background effects for elements.

- CSS background-color

The background-color property specifies the background color of an element.

Example:

The background color of a page is set like this:

```
body {  
  background-color: lightblue;  
}
```

You can set the background color for any HTML elements:

Here, the <h1>, <p>, and <div> elements will have different background colors:

```
h1 {  
  background-color: green;  
}
```

```
div {  
  background-color: lightblue;  
}
```

```
p {  
  background-color: yellow;  
}
```

- CSS Background Image

The background-image property specifies an image to use as the background of an element. By default, the image is repeated so it covers the entire element.

Set the background image for a page:

```
body {  
  background-image: url("paper.gif");  
}
```

Note: When using a background image, use an image that does not disturb the text.

The background image can also be set for specific elements, like the <p> element:

```
p {  
  background-image: url("paper.gif");  
}
```

- CSS background-repeat

By default, the background-image property repeats an image both horizontally and vertically.

Some images should be repeated only horizontally or vertically, or they will look strange, like this:

```
body {  
  background-image: url("gradient_bg.png");  
}
```

If the image above is repeated only horizontally (background-repeat: repeat-x;), the background will look better:

```
body {  
  background-image: url("gradient_bg.png");  
  background-repeat: repeat-x;  
}
```

- CSS background-repeat: no-repeat

Showing the background image only once is also specified by the background-repeat property:

Show the background image only once:

```
body {  
  background-image: url("img_tree.png");  
  background-repeat: no-repeat;  
}
```

In the example above, the background image is placed in the same place as the text. We want to change the position of the image, so that it does not disturb the text too much.

- CSS background-attachment

The background-attachment property specifies whether the background image should scroll or be fixed (will not scroll with the rest of the page):

Example

Specify that the background image should be fixed:

```
body {  
  background-image: url("img_tree.png");  
  background-repeat: no-repeat;  
  background-position: right top;  
  background-attachment: fixed;  
}
```

Example

Specify that the background image should scroll with the rest of the page:

```
body {  
  background-image: url("img_tree.png");  
  background-repeat: no-repeat;  
  • background-position: right top;  
  background-attachment: scroll;  
}
```

- CSS Background Shorthand

To shorten the code, it is also possible to specify all the background properties in one single property. This is called a shorthand property.

Instead of writing:

```
body {  
  background-color: #ffffff;  
  background-image: url("img_tree.png");  
  background-repeat: no-repeat;  
  background-position: right top;  
}
```

You can use the shorthand property background:

Example

Use the shorthand property to set the background properties in one declaration:

```
body {
```

```
background: #ffffff url("img_tree.png") no-repeat right top;
}
```

When using the shorthand property the order of the property values is:

- background-color
- background-image
- background-repeat
- background-attachment
- background-position

It does not matter if one of the property values is missing, as long as the other ones are in this order. Note that we do not use the background-attachment property in the examples above, as it does not have a value.

8 .CSS Borders

The CSS border properties allow you to specify the style, width, and color of an element's border.

- CSS Border Style

The **border-style** property specifies what kind of border to display.

The following values are allowed:

- dotted - Defines a dotted border
- dashed - Defines a dashed border
- solid - Defines a solid border
- double - Defines a double border
- groove - Defines a 3D grooved border. The effect depends on the border-color value
- ridge - Defines a 3D ridged border. The effect depends on the border-color value
- inset - Defines a 3D inset border. The effect depends on the border-color value
- outset - Defines a 3D outset border. The effect depends on the border-color value
- none - Defines no border
- hidden - Defines a hidden border

The border-style property can have from one to four values (for the top border, right border, bottom border, and the left border).

Example

Demonstration of the different border styles:

```
p.dotted {border-style: dotted;}
p.dashed {border-style: dashed;}
```

```
p.solid {border-style: solid;}
p.double {border-style: double;}
p.groove {border-style: groove;}
p.ridge {border-style: ridge;}
p.inset {border-style: inset;}
p.outset {border-style: outset;}
p.none {border-style: none;}
p.hidden {border-style: hidden;}
p.mix {border-style: dotted dashed solid double;}
```

- CSS Border Width

The **border-width** property specifies the width of the four borders.

The width can be set as a specific size (in px, pt, cm, em, etc) or by using one of the three pre-defined values: thin, medium, or thick:

Example

Demonstration of the different border widths:

```
p.one {
  border-style: solid;
  border-width: 5px;
}

p.two {
  border-style: solid;
  border-width: medium;
}

p.three {
  border-style: dotted;
  border-width: 2px;
}

p.four {
  border-style: dotted;
  border-width: thick;
}
```

Specific Side Widths:

The border-width property can have from one to four values (for the top border, right border, bottom border, and the left border):

```
p.one {
  border-style: solid;
```

```

border-width: 5px 20px; /* 5px top and bottom, 20px on the sides */
}

p.two {
border-style: solid;
border-width: 20px 5px; /* 20px top and bottom, 5px on the sides */
}

p.three {
border-style: solid;
border-width: 25px 10px 4px 35px; /* 25px top, 10px right, 4px bottom and 35px left */
}

```

- CSS Border Color

The **border-color** property is used to set the color of the four borders.

Note: If border-color is not set, it inherits the color of the element.

```

p.one {
border-style: solid;
border-color: red;
}

p.two {
border-style: solid;
border-color: green;
}

p.three {
border-style: dotted;
border-color: blue;
}

```

Specific Side Colors

The border-color property can have from one to four values (for the top border, right border, bottom border, and the left border).

```

p.one {
border-style: solid;
border-color: red green blue yellow; /* red top, green right, blue bottom and yellow left */
}

```

- CSS Border Sides

From the examples on the previous pages, you have seen that it is possible to specify a different border for each side.

In CSS, there are also properties for specifying each of the borders (top, right, bottom, and left):

```
p {  
  border-top-style: dotted;  
  border-right-style: solid;  
  border-bottom-style: dotted;  
  border-left-style: solid;  
}
```

So, here is how it works:

1. If the border-style property has four values:
 - border-style: dotted solid double dashed;
 - top border is dotted
 - right border is solid
 - bottom border is double
 - left border is dashed
2. If the border-style property has three values:
 - border-style: dotted solid double;
 - top border is dotted
 - right and left borders are solid
 - bottom border is double
3. If the border-style property has two values:
 - border-style: dotted solid;
 - top and bottom borders are dotted
 - right and left borders are solid
4. If the border-style property has one value:
 - border-style: dotted;
 - all four borders are dotted

- CSS Border - Shorthand Property

Like you saw in the previous page, there are many properties to consider when dealing with borders.

To shorten the code, it is also possible to specify all the individual border properties in one property.

The border property is a shorthand property for the following individual border properties:

border-width

border-style (required)

border-color

Example

```
p {  
  border: 5px solid red;  
}
```

You can also specify all the individual border properties for just one side:

Left Border

```
p {  
  border-left: 6px solid red;  
}
```

Bottom Border

```
p {  
  border-bottom: 6px solid red;  
}
```

- CSS Rounded Borders

The border-radius property is used to add rounded borders to an element:

```
p {  
  border: 2px solid red;  
  border-radius: 5px;  
}
```

9 .CSS Height, Width and Max-width

The CSS height and width properties are used to set the height and width of an element.

The CSS max-width property is used to set the maximum width of an element.

The height and width properties do not include padding, borders, or margins. It sets the height/width of the area inside the padding, border, and margin of the element.

The height and width properties may have the following values:

- auto - This is default. The browser calculates the height and width
- length - Defines the height/width in px, cm, etc.
- % - Defines the height/width in percent of the containing block
- initial - Sets the height/width to its default value
- inherit - The height/width will be inherited from its parent value

```
div {  
  height: 200px;  
  width: 50%;  
  background-color: powderblue;  
}
```

Setting max-width

The max-width property is used to set the maximum width of an element.

The max-width can be specified in length values, like px, cm, etc., or in percent (%) of the containing block, or set to none (this is default. Means that there is no maximum width).

The problem with the <div> above occurs when the browser window is smaller than the width of the element (500px). The browser then adds a horizontal scrollbar to the page.

Using max-width instead, in this situation, will improve the browser's handling of small windows.

Note: If you for some reason use both the width property and the max-width property on the same element, and the value of the width property is larger than the max-width property; the max-width property will be used (and the width property will be ignored).

10 .CSS Padding

The CSS padding properties are used to generate space around an element's content, inside of any defined borders.

With CSS, you have full control over the padding. There are properties for setting the padding for each side of an element (top, right, bottom, and left).

CSS has properties for specifying the padding for each side of an element:

- padding-top

- padding-right
- padding-bottom
- padding-left

All the padding properties can have the following values:

- length - specifies a padding in px, pt, cm, etc.
- % - specifies a padding in % of the width of the containing element
- inherit - specifies that the padding should be inherited from the parent element

```
div {
padding-top: 50px;
padding-right: 30px;
padding-bottom: 50px;
padding-left: 80px;
}
```

Padding - Shorthand Property

If the padding property has four values:

- padding: 25px 50px 75px 100px;
- top padding is 25px
- right padding is 50px
- bottom padding is 75px
- left padding is 100px

If the padding property has three values:

- padding: 25px 50px 75px;
- top padding is 25px
- right and left paddings are 50px
- bottom padding is 75px

If the padding property has two values:

- padding: 25px 50px;
- top and bottom paddings are 25px
- right and left paddings are 50px

If the padding property has one value:

- padding: 25px;
- all four paddings are 25px

Padding and Element Width:

The CSS width property specifies the width of the element's content area. The content area is the portion inside the padding, border, and margin of an element (the box model).

So, if an element has a specified width, the padding added to that element will be added to the total width of the element. This is often an undesirable result.

Example

Here, the <div> element is given a width of 300px. However, the actual width of the <div> element will be 350px (300px + 25px of left padding + 25px of right padding):

```
div {  
  width: 300px;  
  padding: 25px;  
}
```

To keep the width at 300px, no matter the amount of padding, you can use the box-sizing property. This causes the element to maintain its actual width; if you increase the padding, the available content space will decrease.

Example

Use the box-sizing property to keep the width at 300px, no matter the amount of padding:

```
div {  
  width: 300px;  
  padding: 25px;  
  box-sizing: border-box;  
}
```

11 .CSS Margins

- CSS Margins

The CSS margin properties are used to create space around elements, outside of any defined borders.

With CSS, you have full control over the margins. There are properties for setting the margin for each side of an element (top, right, bottom, and left).

CSS has properties for specifying the margin for each side of an element:

- margin-top
- margin-right
- margin-bottom

- margin-left

All the margin properties can have the following values:

- auto - the browser calculates the margin
- length - specifies a margin in px, pt, cm, etc.
- % - specifies a margin in % of the width of the containing element
- inherit - specifies that the margin should be inherited from the parent element

Tip: Negative values are allowed.

Margin - Shorthand Property

To shorten the code, it is possible to specify all the margin properties in one property.

The margin property is a shorthand property for the following individual margin properties:

- margin-top
- margin-right
- margin-bottom
- margin-left

So, here is how it works:

margin: 25px 50px 75px 100px;

- top margin is 25px
- right margin is 50px
- bottom margin is 75px
- left margin is 100px

margin: 25px 50px 75px;

- top margin is 25px
- right and left margins are 50px
- bottom margin is 75px

margin: 25px 50px;

- top and bottom margins are 25px
- right and left margins are 50px

margin: 25px;

- all four margins are 25px

The auto Value

You can set the margin property to auto to horizontally center the element within its container.

The element will then take up the specified width, and the remaining space will be split equally between the left and right margins.

```
div {
```

```
width: 300px;
margin: auto;
border: 1px solid red;
}
```

The inherit Value

This example lets the left margin of the <p class="ex1"> element be inherited from the parent element (<div>):

```
div {
  border: 1px solid red;
  margin-left: 100px;
}

p.ex1 {
  margin-left: inherit;
}
```

- CSS Margin Collapse

Top and bottom margins of elements are sometimes collapsed into a single margin that is equal to the largest of the two margins.

This does not happen on left and right margins! Only top and bottom margins!

Look at the following example:

```
h1 {
  margin: 0 0 50px 0;
}

h2 {
  margin: 20px 0 0 0;
}
```

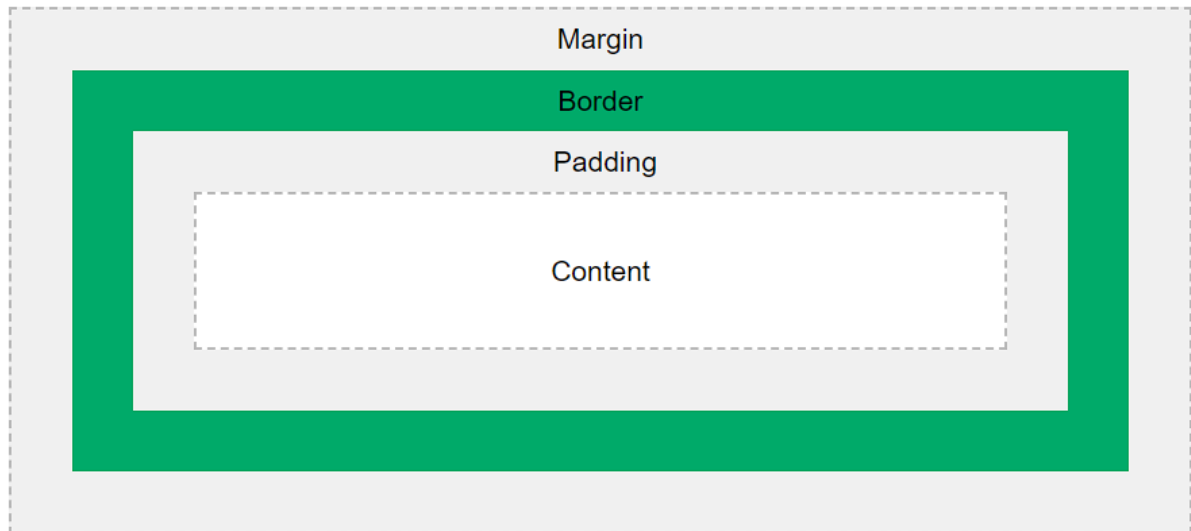
In the example above, the <h1> element has a bottom margin of 50px and the <h2> element has a top margin set to 20px.

Common sense would seem to suggest that the vertical margin between the <h1> and the <h2> would be a total of 70px (50px + 20px). But due to margin collapse, the actual margin ends up being 50px.

12 .CSS Box Model

In CSS, the term "box model" is used when talking about design and layout.

The CSS box model is essentially a box that wraps around every HTML element. It consists of: margins, borders, padding, and the actual content. The image below illustrates the box model:



- Content - The content of the box, where text and images appear
- Padding - Clears an area around the content. The padding is transparent
- Border - A border that goes around the padding and content
- Margin - Clears an area outside the border. The margin is transparent

The total width of an element should be calculated like this:

Total element width = width + left padding + right padding + left border + right border + left margin + right margin

The total height of an element should be calculated like this:

Total element height = height + top padding + bottom padding + top border + bottom border + top margin + bottom margin

13 .CSS Text

CSS has a lot of properties for formatting text.

- Text Color

The default text color for a page is defined in the body selector.

```
body {  
  color: blue;
```

```
}
```

```
h1 {  
  color: green;  
}
```

Text Color and Background Color

```
body {  
  background-color: lightgrey;  
  color: blue;  
}
```

```
h1 {  
  background-color: black;  
  color: white;  
}
```

```
div {  
  background-color: blue;  
  color: white;  
}
```

- CSS Text Alignment

Text Alignment

The text-align property is used to set the horizontal alignment of a text.

A text can be left or right aligned, centered, or justified.

The following example shows center aligned, and left and right aligned text (left alignment is default if text direction is left-to-right, and right alignment is default if text direction is right-to-left):

Example

```
h1 {  
  text-align: center;  
}
```

```
h2 {  
  text-align: left;  
}
```

```
h3 {  
  text-align: right;  
}
```


When the text-align property is set to "justify", each line is stretched so that every line has equal width, and the left and right margins are straight (like in magazines and newspapers):

```
div {  
  text-align: justify;  
}
```

Text Align Last

The text-align-last property specifies how to align the last line of a text.

Align the last line of text in three <p> elements:

```
p.a {  
  text-align-last: right;  
}
```

```
p.b {  
  text-align-last: center;  
}
```

```
p.c {  
  text-align-last: justify;  
}
```

Text Direction

The direction and unicode-bidi properties can be used to change the text direction of an element:

```
p {  
  direction: rtl;  
  unicode-bidi: bidi-override;  
}
```

Vertical Alignment

The vertical-align property sets the vertical alignment of an element.

Example

Set the vertical alignment of an image in a text:

```
img.a {  
  vertical-align: baseline;  
}
```

```
img.b {  
  vertical-align: text-top;  
}
```

```
img.c {  
  vertical-align: text-bottom;  
}
```

```
}
```

```
img.d {  
  vertical-align: sub;  
}
```

```
img.e {  
  vertical-align: super;  
}
```

- CSS Text Decoration

Add a Decoration Line to Text

The `text-decoration-line` property is used to add a decoration line to text.

Tip: You can combine more than one value, like overline and underline to display lines both over and under a text.

```
h1 {  
  text-decoration-line: overline;  
}
```

```
h2 {  
  text-decoration-line: line-through;  
}
```

```
h3 {  
  text-decoration-line: underline;  
}
```

```
p {  
  text-decoration-line: overline underline;  
}
```

Specify a Color for the Decoration Line

The `text-decoration-color` property is used to set the color of the decoration line.

```
h1 {  
  text-decoration-line: overline;  
  text-decoration-color: red;  
}
```

```
h2 {  
  text-decoration-line: line-through;  
}
```

```
text-decoration-color: blue;
}

h3 {
text-decoration-line: underline;
text-decoration-color: green;
}

p {
text-decoration-line: overline underline;
text-decoration-color: purple;
}
```

Specify a Style for the Decoration Line

The text-decoration-style property is used to set the style of the decoration line.

```
h1 {
text-decoration-line: underline;
text-decoration-style: solid;
}
h2 {
text-decoration-line: underline;
text-decoration-style: double;
}
h3 {
text-decoration-line: underline;
text-decoration-style: dotted;
}
p.ex1 {
text-decoration-line: underline;
text-decoration-style: dashed;
}
p.ex2 {
text-decoration-line: underline;
text-decoration-style: wavy;
}
p.ex3 {
text-decoration-line: underline;
text-decoration-color: red;
text-decoration-style: wavy;
}
```

Specify the Thickness for the Decoration Line

The text-decoration-thickness property is used to set the thickness of the decoration line.

```
h1 {  
  text-decoration-line: underline;  
  text-decoration-thickness: auto;  
}
```

```
h2 {  
  text-decoration-line: underline;  
  text-decoration-thickness: 5px;  
}
```

```
h3 {  
  text-decoration-line: underline;  
  text-decoration-thickness: 25%;  
}
```

```
p {  
  text-decoration-line: underline;  
  text-decoration-color: red;  
  text-decoration-style: double;  
  text-decoration-thickness: 5px;  
}
```

The Shorthand Property

The text-decoration property is a shorthand property for:

- text-decoration-line (required)
- text-decoration-color (optional)
- text-decoration-style (optional)
- text-decoration-thickness (optional)

```
h1 {  
  text-decoration: underline;  
}  
h2 {  
  text-decoration: underline red;  
}  
h3 {  
  text-decoration: underline red double;  
}  
p {  
  text-decoration: underline red double 5px;  
}
```

A Small Tip

All links in HTML are underlined by default. Sometimes you see that links are styled with no underline. The text-decoration: none; is used to remove the underline from links, like this:

```
a {  
  text-decoration: none;  
}
```

- CSS Text Transformation

```
p.uppercase {  
  text-transform: uppercase;  
}
```

```
p.lowercase {  
  text-transform: lowercase;  
}
```

```
p.capitalize {  
  text-transform: capitalize;  
}
```

- CSS Text Spacing

Text Indentation

The text-indent property is used to specify the indentation of the first line of a text:

```
p {  
  text-indent: 50px;  
}
```

Letter Spacing

The letter-spacing property is used to specify the space between the characters in a text.

The following example demonstrates how to increase or decrease the space between characters:

```
h1 {  
  letter-spacing: 5px;  
}
```

```
h2 {  
  letter-spacing: -2px;  
}
```

Line Height

The line-height property is used to specify the space between lines:

```
p.small {  
  line-height: 0.8;  
}
```

```
p.big {  
  line-height: 1.8;  
}
```

Word Spacing

The word-spacing property is used to specify the space between the words in a text.

The following example demonstrates how to increase or decrease the space between words:

```
p.one {  
  word-spacing: 10px;  
}
```

```
p.two {  
  word-spacing: -2px;  
}
```

White Space

The white-space property specifies how white-space inside an element is handled.

This example demonstrates how to disable text wrapping inside an element:

```
p {  
  white-space: nowrap;  
}
```

- CSS Text Shadow

The text-shadow property adds shadow to text.

In its simplest use, you only specify the horizontal shadow (2px) and the vertical shadow (2px):

```
h1 {  
  text-shadow: 2px 2px;  
}
```

Next, add a color (red) to the shadow:

```
h1 {  
  text-shadow: 2px 2px red;  
}
```

Then, add a blur effect (5px) to the shadow:

```
h1 {  
  text-shadow: 2px 2px 5px red; }
```

14 .CSS Links

Links can be styled with any CSS property (e.g. color, font-family, background, etc.).

```
a {  
  color: hotpink;  
}
```